

SHOPSPHERE

Decisões de Arquitetura — Aula 06

EVIDÊNCIA — SHOPSPHERESERVICE.JAVA

```
mail.send(  
    "cliente@exemplo.com",  
    "Pedido " + id + " status " + o.status  
);
```

a pista que seguimos →

docs/decisoos_ShopSphere/
ADR-0001 · README-Aula06 · MATRIZ-
DECISAO · JUSTIFICATIVAS-NOTAS

AULA
06

GRUPO SHOPSPHERE

Alice Ferreira do Nascimento
Antonio Silveira Peres Neto
Heloisa Rodrigues Mota
Lucas Almeida Santos

O QUE É O SHOPSPHERE

Marketplace legado: diferentes **vendedores** cadastram produtos, o **cliente** monta o pedido, paga e recebe por serviços de entrega.

“compila e executa, mas é um legado com decisões propositalmente incompletas” — README.md

PONTO DE PARTIDA DA EQUIPE

Time novo no projeto, refatoração aos poucos — sem reescrever tudo de uma vez (*docs/adr/ADR-0001-arquitetura.md*).

o escopo, recortado:

Produtos & Vendedores

Estoque

Carrinho / Pedido

Descontos

Pagamento

Frete

Notificações

Integrações externas

ARQUITETURA DECLARADA PELA EQUIPE ANTERIOR

“monólito modular preparado para microsserviços”

sem evidência de módulos, fronteiras ou trade-offs que sustentem a frase — *docs/arquitetura_inicial.md*

O PROBLEMA REAL

encontrado em `service/ShopSphereService.java`, método `checkout()`

1 · E-MAIL FIXO

A notificação vai sempre para `"cliente@exemplo.com"`, nunca para `o.customer` — nenhum cliente real é avisado do próprio pedido.

3 · HERANÇA INDEVIDA

`PaymentAdapter` extends `CardGatewayLegacy` — a API legada é herdada em vez de encapsulada por composição.

```
CHECKOUT(String id)
```

```
String payResult = payment.cobrar(o.customer, payable);
o.status = payResult.startsWith("00") ? "PAID" : "PAYMENT_ERROR";
if (o.status.equals("PAID")) {
    String freightQuote = freight.cotar("00000-000", 2.0);
}
mail.send(
    "cliente@exemplo.com",
    "Pedido " + id + " status " + o.status
);
```

2 · SEM TRATAMENTO DE FALHA

Nenhum `try/catch` envolve `payment.cobrar(...)` ou `freight.cotar(...)`: uma falha interrompe o fluxo sem registro.

4 · DEPENDÊNCIAS ACOPLADAS

`payment`, `freight`, `mail` e `publisher` nascem com `new` dentro de `ShopSphereService`, não são injetados.

O QUE O SISTEMA PRECISA FAZER

REQUISITOS FUNCIONAIS

- 01 Montar um pedido adicionando produtos, **reservando estoque** na hora da adição (`Product.reserve()`).
- 02 Finalizar a compra: aplicar desconto, cobrar pagamento, cotar frete e **notificar o cliente** (`checkout()`).

REQUISITOS NÃO FUNCIONAIS

- 01 **Segurança** — sem PII em log texto-plano; adapter de pagamento por composição, não herança.
- 02 **Desempenho** — `checkout()` deve responder rápido; hoje é síncrono e local.
- 03 **Confiabilidade** — falha em pagamento/frete não pode deixar o pedido inconsistente nem calar a notificação.
- 04 **Manutenibilidade** — `ShopSphereService` deve receber dependências por injeção, não instanciá-las.

sem critérios de aceite formais no legado — docs/requisitos_iniciais.md

3 CAMINHOS POSSÍVEIS

A

Camadas simples (atual)

Monólito em processo único: Main → ShopSphereFacade → ShopSphereService → Repositório/Adapters, chamadas síncronas.

B

Barramento de eventos interno

Evolui o OrderPublisher/OrderObserver já existente — mesmo processo, sem broker externo — para reagir a eventos do pedido em vez de chamadas em sequência.

C

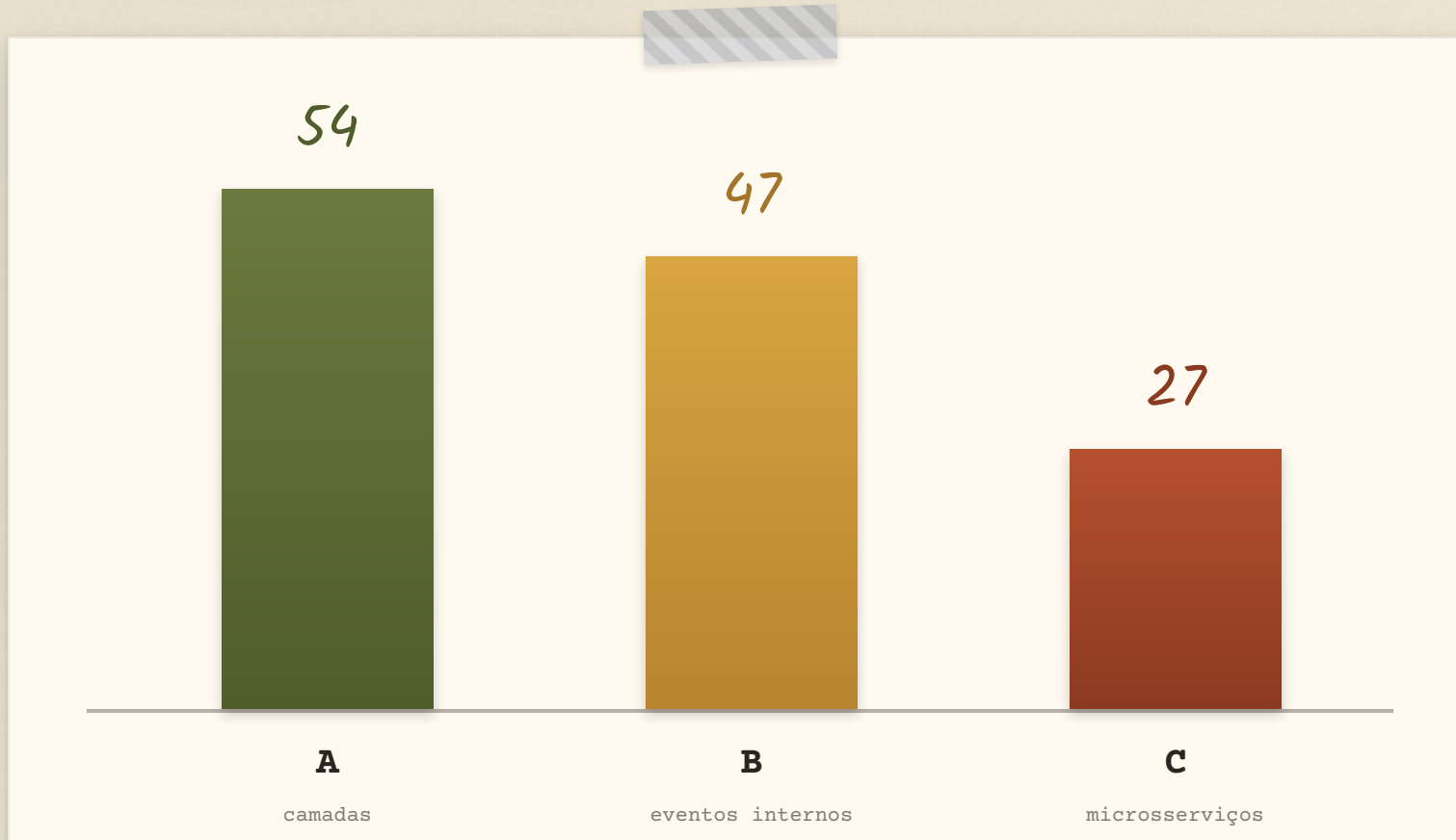
Microserviços por domínio

Catálogo/estoque, pedido, pagamento e frete como serviços independentes, comunicando-se via API.

nenhuma migração é obrigatória só para cumprir a aula — atividades/README-Aula-06.md

MATRIZ DE DECISÃO

critérios: segurança, complexidade operacional, custo de mudança, manutenibilidade, confiabilidade, desempenho — pesos de 1 a 3, notas de 1 a 5.



vencedora ↓

A distância entre A e B é pequena: B pesa em manutenibilidade/confiabilidade, mas perde em complexidade e custo de mudança — os critérios de maior peso agora.

A DECISÃO: MANTER E CORRIGIR



*os problemas são de
implementação, não do estilo em
camadas.*

4 CORREÇÕES INTERNAS, MESMA ARQUITETURA

- ☐ **Injeção de dependência** em ShopSphereService, em vez de instanciar payment/freight/mail/publisher com new.
- ☐ **Composição** no PaymentAdapter, em vez de herdar CardGatewayLegacy.
- ☐ Notificar o **e-mail real do cliente** (o.customer) em checkout(), não um endereço fixo.
- ☐ **Tratar falhas** de payment.cobrar(...) e freight.cotar(...) com try/catch, registrando o que deu errado.

O QUE GANHAMOS, O QUE ABRIMOS MÃO



GANHO

Baixa complexidade operacional e baixo custo de manutenção — o projeto continua rodando com `javac/java -cp`, sem infraestrutura adicional, compatível com o prazo do semestre.

CUSTO

O processo segue síncrono: falha em pagamento ou frete continua exigindo cuidado dentro do próprio `checkout()`, sem a reentrega automática de uma fila de eventos.

TRADE-OFF ACEITO PELO GRUPO

Abrimos mão, por ora, de notificações mais resilientes via fila — não há volume de uso que justifique a infraestrutura hoje. Em troca, o grupo se compromete a corrigir os 4 problemas concretos dentro da própria arquitetura.

PRÓXIMOS PASSOS

- ☐ Corrigir os 4 problemas concretos encontrados em checkout() e no PaymentAdapter.
- ☐ Registrar a evidência da atividade no GitHub – issue, branch e PR.
- ☐ Revisitar a Alternativa B (barramento de eventos interno) se o volume de pedidos crescer de verdade.

Obrigado!

Alice Ferreira do Nascimento · Antonio Silveira
Peres Neto
Heloisa Rodrigues Mota · Lucas Almeida Santos

ADR-0001

README-Aula06

MATRIZ-DECISAO

JUSTIFICATIVAS-NOTAS

docs/decisoes_ShopSphere/